

DAY 2 SALESFORCE TRAINING

24PA5A5708

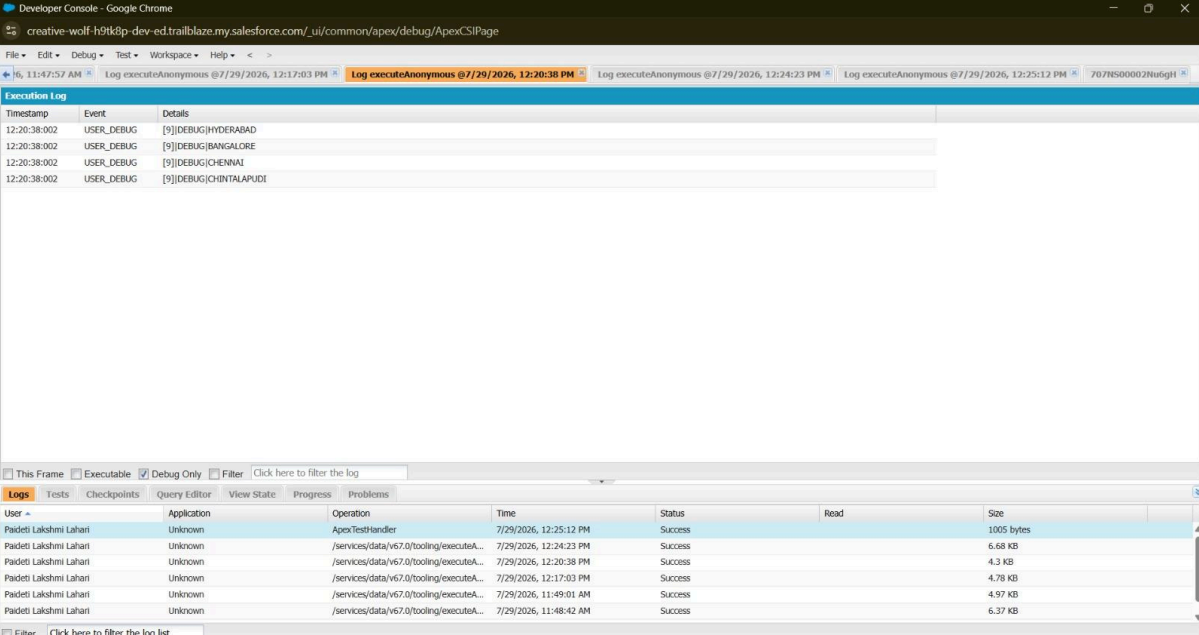
P.Lakshmi Lahari

Block -1

Task 1 — Collections drill

List of 5 city names, loop through, print each in uppercase

```
List<String> cities = new List<String>{  
    'Hyderabad',  
    'Bangalore',  
    'Chennai',  
    'Mumbai',  
    'Delhi'  
};  
for(String city : cities){  
    System.debug(city.toUpperCase());  
}
```



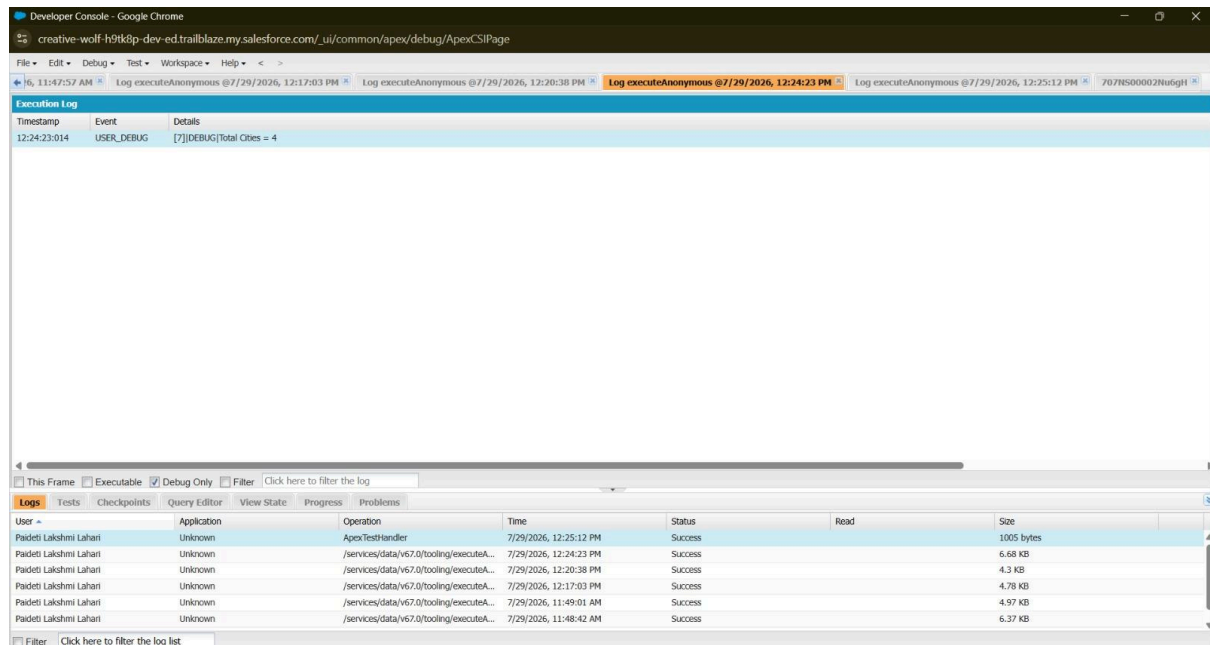
The screenshot shows the Salesforce Developer Console interface. The top section displays the 'Execution Log' with a table of events. The bottom section shows the 'Logs' table with columns for User, Application, Operation, Time, Status, Read, and Size.

Timestamp	Event	Details
12:20:38:002	USER_DEBUG	[9]DEBUG HYDERABAD
12:20:38:002	USER_DEBUG	[9]DEBUG BANGALORE
12:20:38:002	USER_DEBUG	[9]DEBUG CHENNAI
12:20:38:002	USER_DEBUG	[9]DEBUG CHINTALAPUDI

User	Application	Operation	Time	Status	Read	Size
Paideti Lakshmi Lahari	Unknown	ApexTestHandler	7/29/2026, 12:25:12 PM	Success		1005 bytes
Paideti Lakshmi Lahari	Unknown	/services/data/v67.0/tooling/executeA...	7/29/2026, 12:24:23 PM	Success		6.68 KB
Paideti Lakshmi Lahari	Unknown	/services/data/v67.0/tooling/executeA...	7/29/2026, 12:20:38 PM	Success		4.3 KB
Paideti Lakshmi Lahari	Unknown	/services/data/v67.0/tooling/executeA...	7/29/2026, 12:17:03 PM	Success		4.78 KB
Paideti Lakshmi Lahari	Unknown	/services/data/v67.0/tooling/executeA...	7/29/2026, 11:49:01 AM	Success		4.97 KB
Paideti Lakshmi Lahari	Unknown	/services/data/v67.0/tooling/executeA...	7/29/2026, 11:48:42 AM	Success		6.37 KB

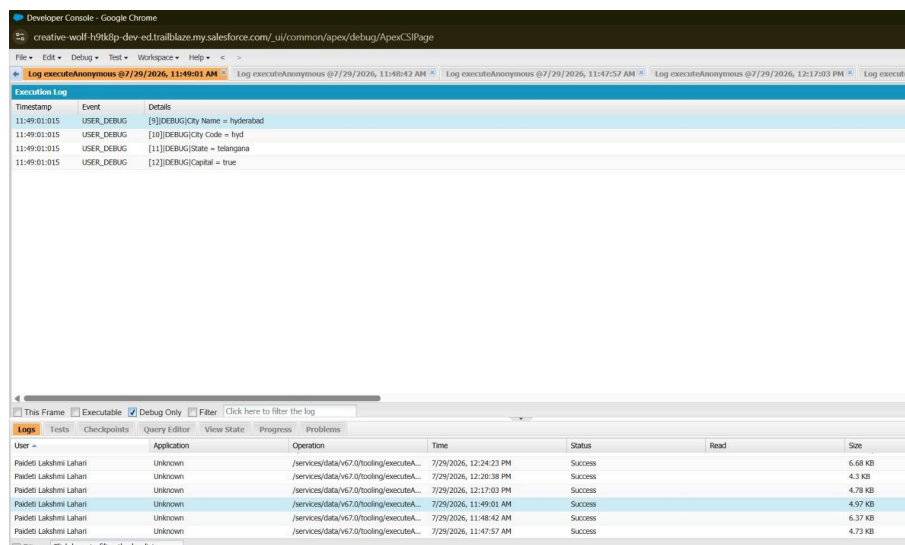
Set from a SOQL query of your custom object (no duplicates), print the size

```
Set<Id> cityIds = new Set<Id>();
for(City__c c : [SELECT Id FROM City__c]){
    cityIds.add(c.Id);
}
System.debug('Set Size = ' + cityIds.size());
```



Map from a SOQL query, look up one record by Id, print a field value

```
Map<Id, City__c> cityMap = new Map<Id, City__c>([ SELECT Id, Name FROM City__c ]);
Id onelId = cityMap.keySet().iterator().next();
System.debug('City Name = ' + cityMap.get(onelId).Name);
```



Block -2

Task 2 — Break it, then fix it

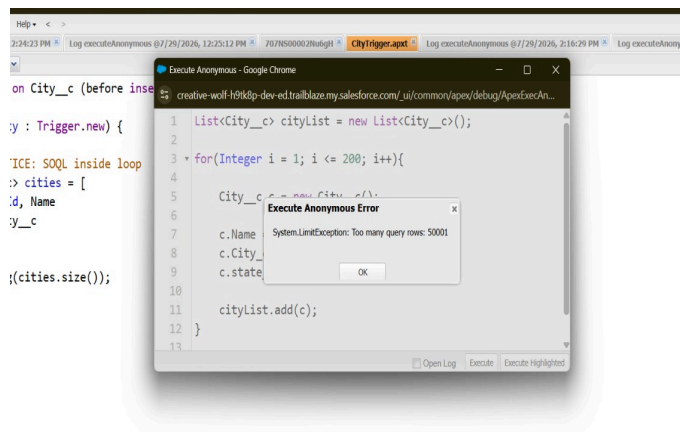
Write a bad trigger on purpose: SOQL or DML inside a for loop.

```
trigger CityTrigger on City__c (before insert) {

    for (City__c city : Trigger.new) {

        // BAD PRACTICE: SOQL inside loop
        List<City__c> cities = [
            SELECT Id, Name
            FROM City__c
        ];

        System.debug(cities.size());
    }
}
```



Write a good trigger on purpose: SOQL or DML inside a for loop.

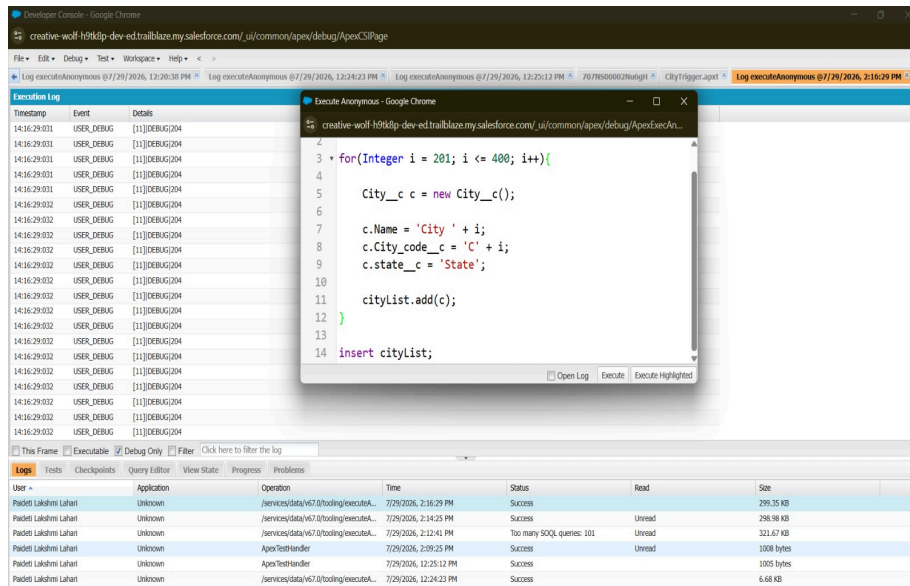
```
trigger CityTrigger on City__c (before insert) {

    // GOOD PRACTICE: SOQL outside the loop
    List<City__c> cities = [
        SELECT Id, Name
        FROM City__c
    ];

    for (City__c city : Trigger.new) {

        System.debug(cities.size());
    }
}
```

```
}
}
```

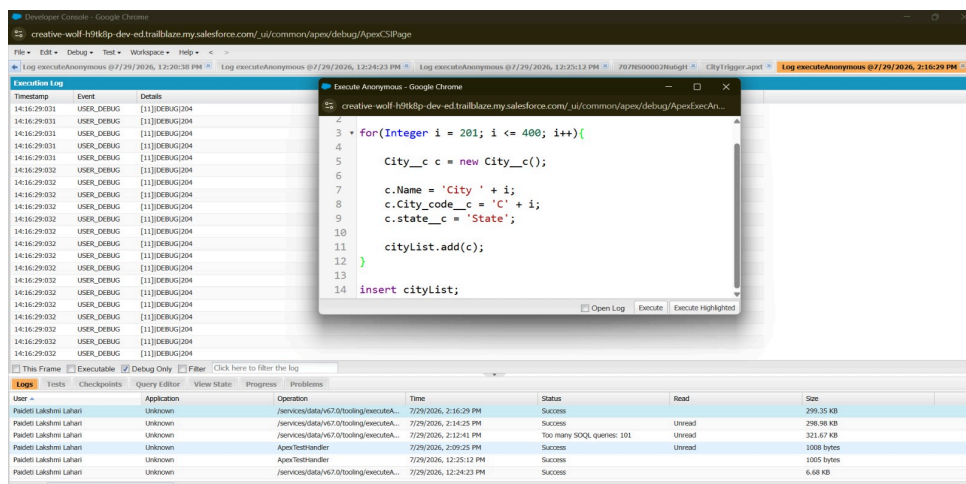


Insert 200 records at once (Data Import) and watch it hit a governor limit.

```

List<City__c> cityList = new List<City__c>();
for(Integer i = 1; i <= 200; i++){
    City__c c = new City__c();
    c.Name = 'City ' + i;
    c.City_code__c = 'C' + i;
    c.state__c = 'State';
    cityList.add(c);
}
insert cityList;

```



Block-3

Asynchronous Apex — Introduction

Task 3 — One @future method

```
public class CityFutureHandler {

    @future
    public static void updateCity(Id cityId) {

        // Query the City record
        City__c city = [
            SELECT Id, Name, state__c
            FROM City__c
            WHERE Id = :cityId
            LIMIT 1
        ];

        // Update the State field
        city.state__c = 'Updated State';

        update city;

        System.debug('City Updated Successfully');
    }
}
```

Call the Future Method

```
City__c city = [
    SELECT Id
    FROM City__c
    LIMIT 1
];

CityFutureHandler.updateCity(city.Id);

System.debug('Future Method Called');
```

Difference between a Future method and Queueable Apex?

Future Method: Used for simple asynchronous operations. It accepts only primitive parameters and cannot chain other jobs.

Queueable Apex: More flexible and powerful. It supports complex data types, job chaining, and better monitoring through Apex Jobs. It is the preferred choice for most new asynchronous Apex implementations.

Why would you need Batch Apex instead of a normal trigger?

Batch Apex is used to process large volumes of records that would exceed the governor limits of a normal trigger. It processes records in smaller batches, making it efficient and reliable for large data operations.

Block 4: LWC — Component Communication

Task 4 — Parent-to-child and child-to-parent

Parent → Child: Parent passes a record Id to a child via a public `@api` property. Child displays a field from that record.

Child → Parent: Button in the child fires a custom event. Parent listens and updates something on screen when it fires

Zip file:

https://drive.google.com/file/d/1VLcbiuyuENX0dJHXJJ_d_J8R0A--Aynv/view?usp=sharing

